

«You cannot step twice into the same river.» – *Heraclitus*

1. Karnaugh Maps

a) Generate the Function

- 1. Compute¹ the SHA-256 *hash* h of string $s = \text{«DM Fall 2025 HW4»}$ (without quotes).
- 2. Split h into eight 32-bit blocks: $h = h_0 \parallel h_1 \parallel \dots \parallel h_7$ where each h_i is 32 bits.
- 3. XOR all blocks together: $d = h_0 \oplus h_1 \oplus \dots \oplus h_7$.
- 4. Apply the mask: $w = d \oplus 0x71be8976$, giving the 32-bit result $w = (w_0w_1\dots w_{31})_2$.

The resulting bit string w encodes the 5-variable *boolean function* $f^{(5)}$: bit w_0 (MSB) represents $f(0, 0, 0, 0, 0)$, and bit w_{31} (LSB) represents $f(1, 1, 1, 1, 1)$.

Check: Hash h ends with ...01100010; the value d (before masking) begins with 0000...

b) Trying to master K-Map

Construct a 5-variable *Karnaugh map* for your function using the template below:

		E				E			
		000	001	011	010	110	111	101	100
A	00								
	01								
	11								
	10								
		D				C			



¹You can use any programming language, tool or online service to compute SHA-256 hashes.

c) Find Minimal Forms

1. Find *minimal* DNF and CNF for f using your K-map.
2. Count all *prime implicants*. (PI).
3. If inputs where $A \wedge B \wedge C = 1$ are *don't-care conditions*², how do the minimal forms change?

d) Analysis and Limitations

1. How many fewer *literals* does your minimal DNF use compared to full minterm expansion?
2. Which is more compact for your function: DNF or CNF? Why?

2. Boolean Function Analysis

Consider the following four functions³:

1. $f_1 = f_{47541}^{(4)}$
2. $f_2 = \sum m(1, 4, 5, 6, 8, 12, 13)$
3. $f_3 = f_{51011}^{(4)} \oplus f_{40389}^{(4)}$
4. $f_4 = A\overline{B}D + \overline{A}\overline{C}D + \overline{B}C\overline{D} + A\overline{C}D$

a) Boolean Function Decomposition

1. Decompose to SoP using *Shannon expansion* for function 4. (without constructing the truth table).
2. Construct PoS for function 4. from the known SoP.

b) K-Maps

Draw and fill the K-map for each function.

c) Prime Implicants

1. List all *implicants* of function 4.
2. Using the K-map, find all *prime implicants* of function 4. (Blake Canonical Form)⁴
3. Using the *Quine-McCluskey method*, find all *prime implicants* of function 4. (Blake Canonical Form)
4. Determine which of them are *essential* (EPI).

d) Minimal DNF and CNF

1. Using *Petrick's method* (on the already obtained result of the Quine-McCluskey method) find the minimal DNF of function 4.
2. Using *algebraic method* find the minimal CNF of function 4.

e) Algebraic Normal Form

1. Construct the ANF for function 1. using *Pascal's method*.
2. Construct the ANF for function 2. using *triangle method*.
3. Construct the ANF for function 4. using *K-map*.

²Don't-care conditions are input combinations whose output values don't matter, allowing flexibility in minimization. They can be treated as either 0 or 1 to produce smaller expressions.

³Notation: $f_k^{(n)}$ is the k -th Boolean function of n variables, where k is the decimal truth table value, which, represented in binary as $(f_0 f_1 \dots f_{2^n-1})_2$, corresponds to values of f : MSB f_0 is $f(0, \dots, 0)$, LSB f_{2^n-1} is $f(1, \dots, 1)$.

⁴Reduced DNF - Blake Canonical Form

f) Zhegalkin Polynomial

1. Identify the *algebraic degree* of each function from part (e).
2. Which function has the highest degree? Is it *maximal* for this number of variables?
3. Prove that $\deg(f \oplus g) \leq \max(\deg(f), \deg(g))$. Find an example with strict inequality.

3. Reed–Muller Codes

(optional, +1 extra point)

In 1971, Mariner 9 transmitted the first close-up Mars images using Reed–Muller codes. These error-correcting codes are built from Boolean functions with restricted algebraic degree.

The Reed–Muller code $RM(r, m)$ consists of all Boolean functions $f : \mathbb{F}_2^m \rightarrow \mathbb{F}_2$ with $\deg(f) \leq r$.

Parameters: length $n = 2^m$, dimension $k = \sum_{i=0}^r \binom{m}{i}$, minimum *Hamming distance*⁵ $d = 2^{m-r}$.

Generator matrix construction: Rows correspond to *monomials* of degree $\leq r$. Columns correspond to inputs $(x_1, \dots, x_m) \in \mathbb{F}_2^m$. Entry (i, j) is the value of monomial i evaluated at input j .

Example: $RM(1, 2)$ has monomials $\{1, x_1, x_2\}$ and inputs $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$:

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

Message $(1, 0, 1)$ encodes to $(1, 0, 1, 0)$ via $c = uG$.

a) Code Construction

Consider $RM(1, 3)$:

1. Compute the code parameters: length n , dimension k , minimum distance d , and error correction capability $t = \lfloor \frac{d-1}{2} \rfloor$.
2. Construct the 4×8 generator matrix G . Rows correspond to monomials $\{1, x_1, x_2, x_3\}$. Columns are ordered: $(0, 0, 0), (0, 0, 1), \dots, (1, 1, 1)$, where column i (for $i = 1, \dots, 8$) corresponds to the binary representation of $i - 1$.
3. Encode the message $u = (1, 0, 1, 1)$ into codeword $c = uG$.

b) Single-Error Correction

1. Flip *one* bit at position $i \in \{1, \dots, 8\}$ of your codeword c to simulate a transmission error, obtaining received word $r = (r_1, \dots, r_8)$.
2. Apply majority-logic decoding to recover the ANF coefficients (a_0, a_1, a_2, a_3) :
 - For each a_j where $j > 0$: identify all pairs of positions (p, q) whose input vectors differ only in variable x_j . Each pair gives a vote $v = r_p \oplus r_q$. Set $a_j = \text{majority}(v_1, v_2, \dots)$.
 - For a_0 : after determining a_1, a_2, a_3 , compute $a_0 = r_1 \oplus (a_1 \cdot 0) \oplus (a_2 \cdot 0) \oplus (a_3 \cdot 0) = r_1$, since position 1 corresponds to input $(0, 0, 0)$.⁶

⁵The *Hamming distance* between two codewords is the number of positions in which they differ.

3. Verify that the decoded message (a_0, a_1, a_2, a_3) equals your original $\mathbf{u} = (1, 0, 1, 1)$.
4. Show the detailed computation for coefficient a_1 : list all pairs differing in x_1 , compute each XOR vote, and determine the majority.

⁶In case of tied votes, choose 0 as the default value. Note that with a single error, ties should not occur for $\text{RM}(1, 3)$.